

CPF | Core Platform Framework

Business Platform Documentation

CPF 데이터베이스표준서

Oracle·PostgreSQL·MariaDB 의 Canonical Schema 와 Migration·Rollback·Runtime 검증 기준을 정의합니다.

문서 기준	Documentation Harness v2.1.0
기준 Source	master 054d894b47f4be8323439dc6f9e58b7d8b60fe54
기준일	2026-08-26

이 문서는 최신 Source 의 공개 계약과 운영 경계를 기준으로 작성하며, 실행하지 않은 검증을 성공으로 기록하지 않습니다.

목차

역할과 질문을 기준으로 필요한 절을 빠르게 찾습니다.

1. DB3 적용 범위

- 1.1 Oracle · PostgreSQL · MariaDB
- 1.2 Canonical DB Source
- 1.3 Vendor 차이 원칙

2. Naming · Schema · Table · Column

- 2.1 Naming
- 2.2 Schema
- 2.3 Table/Column

3. Datatype · Key · Constraint

- 3.1 Datatype
- 3.2 PK/FK
- 3.3 Unique/Check

4. Index · Identity

- 4.1 Index
- 4.2 Sequence/Identity

5. Audit · Lock · Batch Claim

- 5.1 Audit/History
- 5.2 Lock/Concurrency
- 5.3 Batch Claim

6. Fresh Init · Migration · Seed

- 6.1 Fresh Initialization
- 6.2 Migration
- 6.3 Seed

7. Upgrade · Rollback · Recovery

- 7.1 Upgrade
- 7.2 Rollback/Recovery

8. Runtime Query · Performance

- 8.1 Runtime Query
- 8.2 Performance
- 8.3 Lock/Query 영향

9. Schema Mismatch · Generator Binding

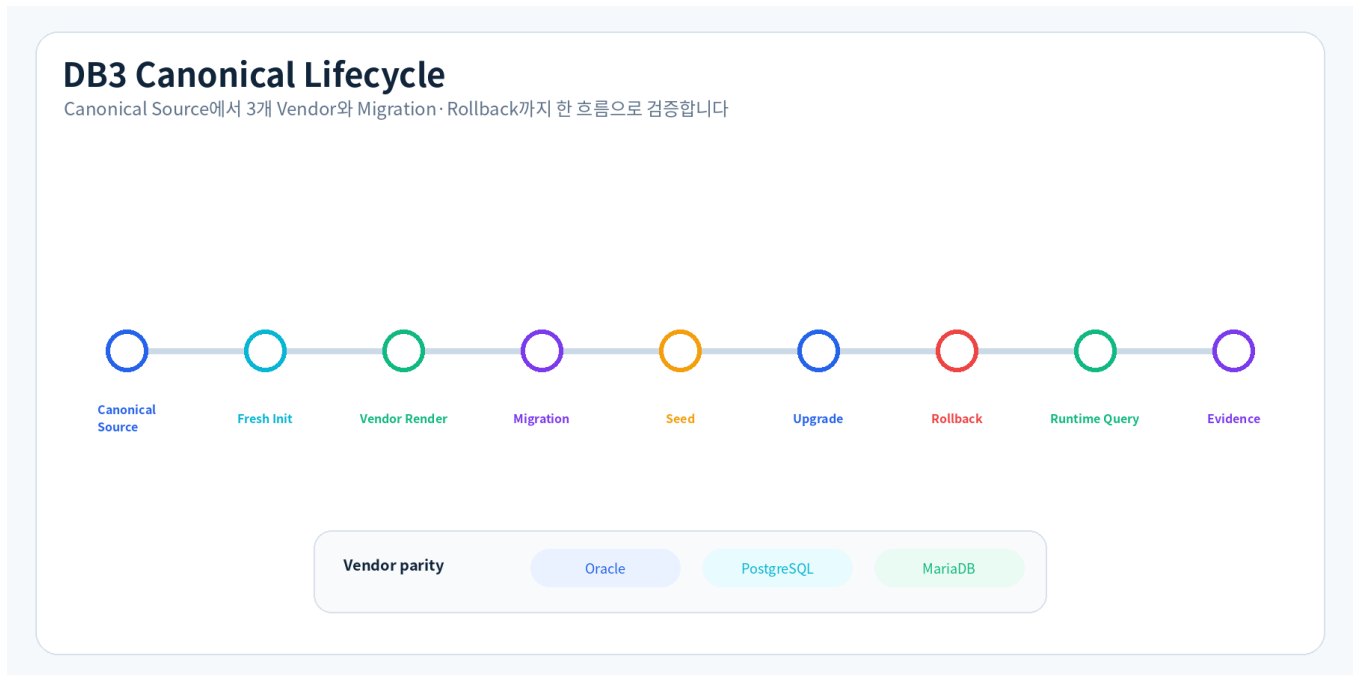
- 9.1 Schema Mismatch
- 9.2 Generator/DB Binding
- 9.3 Vendor Difference

10. Validation · Evidence

- 10.1 검증 항목
- 10.2 Runtime Evidence
- 10.3 Rollback/Recovery Evidence

1. DB3 적용 범위

그림 1-1. DB3 Canonical Lifecycle



Canonical Source 에서 Oracle·PostgreSQL·MariaDB 와 Migration·Rollback·Evidence 까지 연결합니다.

1.1 Oracle·PostgreSQL·MariaDB

CPF의 공식 DB Vendor는 Oracle, PostgreSQL, MariaDB입니다. 하나의 Canonical DB 의미를 기준으로 Vendor 별 DDL·runtime query 차이를 명시적으로 render/검증합니다.

공식 DB Vendor는 Oracle·PostgreSQL·MariaDB만 사용하며 세 Vendor의 DDL·Migration·Rollback·Runtime 동작을 함께 검증합니다.

- Vendor 차이는 datatype·identity·locking·DDL syntax 처럼 실제 동작 차이가 있는 지점에서만 분기합니다.

1.2 Canonical DB Source

Canonical DB Source는 schema, table, column, constraint, index와 seed 의미의 단일 정보입니다.

Oracle·PostgreSQL·MariaDB DDL은 이 정의에서 파생되어야 합니다.

1.3 Vendor 차이 원칙

Vendor 차이는 datatype, identity, locking, DDL syntax 처럼 실제 동작이 다른 지점에만 둡니다. Vendor 별 편의 때문에 Canonical 업무 의미를 분기하지 않습니다.

Vendor 차이는 datatype, identity, locking, DDL syntax 처럼 실제 동작 차이가 있는 부분에 한정하고 업무 의미를 Vendor 별로 갈라놓지 않습니다.

Canonical 의미를 유지하면서 Vendor 별로 달라져야 하는 datatype·identity·DDL 지점만 비교합니다.

표 1-1. 항목 / Oracle / PostgreSQL / MariaDB

비교·선택·확인에 필요한 항목만 압축해 정리합니다.

항목	Oracle	PostgreSQL	MariaDB
공식 지원	지원	지원	지원
Canonical 의미	공통	공통	공통
Vendor Render	oracle	postgresql	mariadb
검증	DDL·Runtime	DDL·Runtime	DDL·Runtime

[Source 길찾기 cpf-tools/db/vendor/oracle](#)

2. Naming·Schema·Table·Column

2.1 Naming

DB Object 는 Canonical naming 과 데이터 의미를 우선하고 Vendor 별 물리 표현은 renderer 가 담당합니다. Key/Constraint 는 데이터 무결성을 Application 코드에만 의존하지 않게 정의합니다.

- Table/Column 이름은 역할이 드러나는 일관된 규칙을 사용합니다.
- FK·Unique·Check 의 운영 영향과 migration 순서를 함께 검토합니다.

2.2 Schema

Schema 는 DB Owner 와 lifecycle 경계를 표현합니다. Fresh Init, Migration, Upgrade 에서 동일한 Owner 기준을 유지하고 업무 Domain 원장을 다른 Module 이 직접 소유하지 않습니다.

2.3 Table/Column

Table 과 Column 은 Canonical 의미, nullability, default, key 관계를 먼저 정의하고 Vendor 별 DDL 표현은 renderer 가 담당합니다.

Table/Column naming·nullability·default·key 관계의 필수 규칙을 확인합니다.

[Source 길찾기 cpf-tools/db/vendor/oracle](#)

3. Datatype·Key·Constraint

3.1 Datatype

Datatype 은 Canonical 의미를 기준으로 정의하고 Oracle·PostgreSQL·MariaDB 의 표현 차이는 값 범위와 정밀도를 보존하는 범위에서만 분기합니다.

3.2 PK/FK

PK 는 안정적인 row identity 를 제공하고 FK 는 Owner 간 참조 무결성을 명시합니다. 생성·변경·삭제 순서는 Migration 과 Rollback/Recovery 에서 함께 검증합니다.

3.3 Unique/Check

Unique 와 Check 는 Application 검증만으로 대체하지 않고 DB 무결성 계약으로 유지합니다. Vendor 별 문법 차이는 Canonical 의미를 바꾸지 않습니다.

Unique/Check 제약이 DB3 에서 동일한 무결성 결과를 만드는지 비교합니다.

표 3-1. 항목 / Oracle / PostgreSQL / MariaDB

비교·선택·확인에 필요한 항목만 압축해 정리합니다.

항목	Oracle	PostgreSQL	MariaDB
공식 지원	지원	지원	지원
Canonical 의미	공통	공통	공통
Vendor Render	oracle	postgresql	mariadb
검증	DDL·Runtime	DDL·Runtime	DDL·Runtime

DB 무결성 제약의 Canonical 의미와 Vendor 별 표현 제한을 확인합니다.

[Source 길찾기 cpf-tools/db/vendor/oracle](#)

4. Index·Identity

4.1 Index

Index 는 실제 Runtime Query 와 cardinality 를 기준으로 설계합니다. 읽기 성능만 보지 않고 DML 비용, lock 영향, 중복 index 를 함께 검토합니다.

- PK/FK/검색 조건과 execution plan 을 확인합니다.
- Vendor 별 이름 길이·online DDL 제약은 배포 절차에서 확인합니다.

4.2 Sequence/Identity

Sequence/Identity 는 Vendor 별 생성 방식이 달라도 동일한 identity 의미와 동시성 안전성을 유지하도록 renderer 와 runtime query 를 함께 검증합니다.

identity 생성 방식의 DB3 차이와 동시성 검증 기준을 확인합니다.

[Source 길찾기 cpf-tools/db/vendor/oracle](#)

5. Audit·Lock·Batch Claim

5.1 Audit/History

Audit/History 데이터는 누가·언제·무엇을·왜 변경했는지 재구성할 수 있어야 합니다. 업무 원장과 감사 이력의 retention/권한을 분리합니다.

- 민감정보 원문을 감사 로그에 복제하지 않습니다.
- 운영 위험조치에는 reason/approval/result 를 연결합니다.

5.2 Lock/Concurrency

Lock 은 동일 Resource 에 대한 동시 Side Effect 를 제어하는 수단입니다. DB lock, optimistic concurrency, distributed lock 중 소유 범위와 장애 모델에 맞는 방식을 선택합니다.

- lease 기반 lock 은 만료와 fencing token 을 함께 검토합니다.
- lock 대기시간을 transaction timeout 보다 길게 두지 않습니다.

5.3 Batch Claim

Batch Claim 은 다중 Worker 가 같은 대상을 중복 처리하지 않도록 claim 상태, owner instance, lease 만료와 fencing token 을 DB 조건으로 관리합니다.

- claim update 는 현재 owner/lease/fencing 조건을 원자적으로 비교해 stale Worker 의 갱신을 거절합니다.
- Worker 재할당 뒤 이전 fencing token 으로 쓰기가 가능한지 동시성 Test 로 검증합니다.

claim·lease·fencing 컬럼과 원자적 갱신 조건을 확인합니다.

[Source 길찾기](#) `cpf-starters/data/src/main/java/com/cpf/data/lock/api/CpfLockManager.java`

6. Fresh Init·Migration·Seed

6.1 Fresh Initialization

Fresh Initialization 은 Canonical DB Source 를 기준으로 빈 DB/User/Schema 를 준비하고 DB3 Vendor Render 와 Seed 까지 재현 가능하게 구성합니다.

- Fresh Init 은 기존 객체에 의존하지 않는 빈 환경에서 schema·seed 까지 재현해야 합니다.

6.2 Migration

Migration 은 현재 Schema 에서 목표 Schema 로의 전이를 정의하며 순서, 기존 데이터, index/FK 영향과 실패 시 복구 경로를 함께 검증합니다.

- Migration 은 적용 순서와 기존 데이터 보존, 실패 시 복구 지점을 함께 검증합니다.

6.3 Seed

Seed 는 필수 기준 데이터를 재현 가능하고 재실행 안전하게 반영하며 Vendor 별 결과가 같은 업무 의미를 가져야 합니다.

- Seed 는 재실행해도 중복 기준 데이터나 다른 Vendor 결과를 만들지 않아야 합니다.

DB 변경이 Fresh Init 부터 Runtime Query 까지 닫혔는지 확인합니다.

표 6-1. 단계 / Canonical / Vendor 영향 / 검증/복구

비교·선택·확인에 필요한 항목만 압축해 정리합니다.

단계	Canonical	Vendor 영향	검증/복구
Fresh Init	Canonical schema	Vendor 별 DDL	깨끗한 초기화
Migration	변경 정의	문법/락 차이	upgrade 실행

단계	Canonical	Vendor 영향	검증/복구
Seed	공통 기준 데이터	DML 차이	재실행 안전성
Rollback/Recovery	역변경/복구	Vendor 별 절차	실패 복구 Evidence
Runtime Query	Repository 의미	SQL 차이	실거래 검증

Source [길찾기 cpf-tools/db/vendor/oracle](#)

7. Upgrade·Rollback·Recovery

7.1 Upgrade

Upgrade 는 Migration, Runtime compatibility, 데이터 보존과 배포 순서를 함께 검증해 구버전/신버전 혼재 구간의 위험을 관리합니다.

- Upgrade 는 구·신 Runtime 혼재 구간과 Schema compatibility 를 포함해 검증합니다.

7.2 Rollback/Recovery

Rollback 가능한 변경은 역전 절차를 검증하고, 역전이 위험하거나 불가능한 변경은 명시적인 Recovery 절차와 판단 기준을 제공합니다.

- 역전 불가 변경은 rollback 성공으로 꾸미지 않고 명시적 recovery 절차를 제공합니다.

DB 변경이 Fresh Init 부터 Runtime Query 까지 단렸는지 확인합니다.

표 7-1. 단계 / Canonical / Vendor 영향 / 검증/복구

비교·선택·확인에 필요한 항목만 압축해 정리합니다.

단계	Canonical	Vendor 영향	검증/복구
Fresh Init	Canonical schema	Vendor 별 DDL	깨끗한 초기화
Migration	변경 정의	문법/락 차이	upgrade 실행
Seed	공통 기준 데이터	DML 차이	재실행 안전성
Rollback/Recovery	역변경/복구	Vendor 별 절차	실패 복구 Evidence
Runtime Query	Repository 의미	SQL 차이	실거래 검증

Source [길찾기 cpf-tools/db/vendor/oracle](#)

8. Runtime Query·Performance

8.1 Runtime Query

Runtime Query 와 Repository/Mapper 는 적용된 Schema 와 동일 계약을 사용합니다. Migration 완료 전에 신규 Query 를 먼저 배포하지 않고 index/lock 영향을 실제 실행에서 확인합니다.

Runtime Query 와 Repository/Mapper 는 적용된 Schema 와 동일 계약을 사용해야 하며 Migration 완료 전 신규 Query 를 먼저 배포하지 않습니다.

8.2 Performance

성능 판단은 TPS 하나가 아니라 latency, DB lock/query, 외부 연계 backlog, Worker 수, resource saturation 을 함께 봅니다. 용량 변경은 장애 격리와 replay 폭증 가능성까지 확인합니다.

- 대량 작업은 샘플 성공이 아니라 최대/경계 규모에서 측정합니다.
- Scale-out 시 shared state 와 idempotency/lease 가 다중 인스턴스를 견뎌야 합니다.

8.3 Lock/Query 영향

Runtime Query 가 lock 을 획득하거나 장시간 유지하는 경우 실행계획, lock wait, timeout 과 대상 row 범위를 함께 측정해 처리량·교착 위험을 확인합니다.

- lease 기반 lock 은 만료와 fencing token 을 함께 검토합니다.
- lock 대기시간을 transaction timeout 보다 길게 두지 않습니다.

실제 Runtime Query 의 index·lock·timeout 영향과 금지 패턴을 확인합니다.

[Source 길찾기 cpf-starters/data/src/main/java/com/cpf/data/lock/api/CpfLockManager.java](#)

9. Schema Mismatch·Generator Binding

9.1 Schema Mismatch

Runtime 이 기대하는 Schema 와 실제 DB 가 다르면 조용히 우회하지 않고 mismatch 를 식별해 Migration/Recovery 또는 배포 중단으로 연결합니다.

9.2 Generator/DB Binding

Generator 가 만든 Domain 의 datasource, migration, repository 설정은 Canonical DB Catalog 를 따라야 합니다. 개별 Domain 이 Vendor 전용 경로를 수동 추가해 parity 를 깨지 않습니다.

Generator 가 만든 Domain 의 DB 설정과 migration 연결은 Canonical DB Catalog 를 따라야 하며 수동 Vendor 전용 설정으로 우회하지 않습니다.

9.3 Vendor Difference

Vendor Difference 검증은 같은 Canonical Schema 가 Oracle·PostgreSQL·MariaDB 에서 동일 constraint·identity·query 의미를 갖는지 비교합니다.

- Vendor 별 DDL/Query 차이는 Canonical 의미를 바꾸지 않는 범위에서만 허용합니다.
 - 같은 Schema change 를 DB3 에 적용해 constraint·identity·runtime query 결과를 비교합니다.
- 같은 Canonical Schema 가 DB3 에서 동일한 제약·identity·query 의미를 유지하는지 비교합니다.

표 9-1. 항목 / Oracle / PostgreSQL / MariaDB

비교·선택·확인에 필요한 항목만 압축해 정리합니다.

항목	Oracle	PostgreSQL	MariaDB
공식 지원	지원	지원	지원
Canonical 의미	공통	공통	공통
Vendor Render	oracle	postgresql	mariadb
검증	DDL·Runtime	DDL·Runtime	DDL·Runtime

[Source](#) [길찾기](#) [cpf-tools/db/vendor/oracle](#)

10. Validation·Evidence

10.1 검증 항목

검증은 Unit/Integration 일부 PASS 가 아니라 정상·오류·경계·부분 실패·UNKNOWN·복구·재실행까지 영향 Lifecycle 을 확인합니다.

- 실행하지 않은 Test 는 PASS 로 기록하지 않습니다.
- Source identity, 명령, ExitCode, Evidence 를 같은 실행 결과에 연결합니다.

10.2 Runtime Evidence

Runtime Evidence 는 실제 DB3 Query 결과, migration history, index/FK/lock 상태와 Runtime Trace 를 포함해야 합니다. SQL 파일 존재만으로 적용 성공을 판정하지 않습니다.

Runtime Evidence 는 실제 DB3 Query/History/Trace 와 적용 migration version 을 확인해 문서상 DDL 존재만으로 성공 처리하지 않습니다.

10.3 Rollback/Recovery Evidence

Rollback/Recovery Evidence 에는 기준 Source, 대상 Vendor, 실행 명령, 변경 전후 상태, ExitCode 와 복구 결과를 같은 실행 기록으로 남깁니다.

- Evidence 에는 실제 rollback 또는 recovery 실행 결과와 복구 후 query 검증을 포함합니다.

정상·실패·복구까지 통과해야 할 시나리오를 확인합니다.

[Source](#) [길찾기](#) [cpf-tools/db/vendor/oracle](#)